

Lab 1

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure: Servers (1) > PostgreSQL 18 > Databases (2) > dvdrental. The main pane shows a SQL query in the 'Query' tab:

```
8 explain (analyze, buffers)
9 select c.first_name, c.last_name, r.rental_date
10 from customer c
11 join rental r on c.customer_id = r.customer_id
12 where c.customer_id = 1;
```

Below the query, the 'Data Output' tab shows the 'QUERY PLAN' for the query. The plan is as follows:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loops
1	Nested Loop	(cost=0.28..359.16 rows=32 width=21)			(actual time=4.146..16.082 rows=32.00 loops=1)		
2	Buffers: shared hit=4 read=149 dirtied=1						
3	-> Index Scan using customer_pkey on customer c	(cost=0.28..8.29 rows=1 width=17)			(actual time=0.982..0.986 rows=1.00 loops=1)		
4	Index Cond: (customer_id = 1)						
5	Index Searches: 1						
6	Buffers: shared hit=3						
7	-> Seq Scan on rental r	(cost=0.00..350.55 rows=32 width=10)			(actual time=3.159..15.079 rows=32.00 loops=1)		
8	Filter: (customer_id = 1)						
9	Rows Removed by Filter: 16012						
10	Buffers: shared hit=1 read=149 dirtied=1						
11	Planning:						
12	Buffers: shared hit=89 read=1 dirtied=1						
13	Planning Time: 5.269 ms						
14	Execution Time: 16.139 ms						

At the bottom, the status bar shows: Total rows: 14, Query complete 00:00:00.106, CRLF, Ln 13, Col 1.

- Identify the Roles:

- + Outer table: customer
- + Inner table: rental

- Nested loop: The customer table returns only 1 row, so nested loop is efficient. Hash join would add unnecessary overhead

- Efficiency: Inner loops = 1: The inner table runs once because the outer table produces only 1 row

Lab 2

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (2)
 - dvdrental
 - postgres
 - Login/Group Roles
 - Tablespaces

Dependencies Dependents Processes dvdrental/postgres@PostgreSQL 18

Query

```

8 explain (analyze, buffers)
9 select c.first_name, c.last_name, r.rental_date
10 from customer c
11 join rental r on c.customer_id = r.customer_id
12 where c.customer_id = 1;

```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

QUERY PLAN
1 Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=2.556..5.104 rows=1000.00 loops=1)
2 Hash Cond: (f.language_id = l.language_id)
3 Buffers: shared hit=88 read=1
4 -> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.039..2.312 rows=1000.00 loops=1)
5 Buffers: shared hit=88
6 -> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=2.482..2.482 rows=6.00 loops=1)
7 Buckets: 1024 Batches: 1 Memory Usage: 9kB
8 Buffers: shared read=1
9 -> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=2.460..2.462 rows=6.00 loops=1)
10 Buffers: shared read=1
11 Planning:
12 Buffers: shared hit=166 read=1
13 Planning Time: 7.979 ms
14 Execution Time: 5.293 ms

Total rows: 14 Query complete 00:00:00.132 CRLF Ln 15, Col 1

- Identify Phases:

- + Hashed (Built) table: language
- + Probed table: film

- MemoryCheck: The Memory Usage is 9 kB

- Hypothesis:

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (2)
 - dvdrental
 - postgres
 - Login/Group Roles
 - Tablespaces

Dependencies Dependents Processes dvdrental/postgres@PostgreSQL 18

Query

```

13 set work_mem = '64kB';
14
15 explain (analyze, buffers)
16 select f.title, l.name as language
17 from film f
18 join language l on f.language_id = l.language_id;

```

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1 of 1

QUERY PLAN
1 Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.100..0.668 rows=1000.00 loops=1)
2 Hash Cond: (f.language_id = l.language_id)
3 Buffers: shared hit=89
4 -> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.046..0.321 rows=1000.00 loops=1)
5 Buffers: shared hit=88
6 -> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.033..0.033 rows=6.00 loops=1)
7 Buckets: 1024 Batches: 1 Memory Usage: 9kB
8 Buffers: shared hit=1
9 -> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.015..0.017 rows=6.00 loops=1)
10 Buffers: shared hit=1
11 Planning Time: 0.559 ms
12 Execution Time: 0.860 ms

Total rows: 12 Query complete 00:00:00.115 CRLF Ln 15, Col 1

With `work_mem = 64kB`, PostgreSQL still chooses hash join because the hash table only requires 9 kB of memory, which fits within the 64 kB limit.

Lab 3

The screenshot shows the pgAdmin 4 interface with a query window open. The query is as follows:

```
20
21 set enable_hashjoin = off;
22
23 explain (analyze, buffers)
24 select f.film_id, f.title, i.inventory_id
25 from film f
26 join inventory i on f.film_id = i.film_id
27 order by f.film_id;
28
```

The query plan (QUERY PLAN) is displayed below the query text:

Step	Operation	Cost	Time	Rows	Width	Loops
1	Merge Join	(cost=412.64..622.36 rows=4581 width=23)	(actual time=7.174..9.581 rows=4581.00 loops=1)			
2	Merge Cond: (f.film_id = i.film_id)					
3	Buffers: shared hit=91 read=22, temp read=10 written=12					
4	-> Index Scan using film_pkey on film f	(cost=0.28..127.33 rows=1000 width=19)	(actual time=0.017..0.410 rows=1000.00 loops=1)			
5	Index Searches: 1					
6	Buffers: shared hit=85					
7	-> Materialize	(cost=412.37..435.27 rows=4581 width=6)	(actual time=7.146..8.148 rows=4581.00 loops=1)			
8	Storage: Memory Maximum Storage: 17kB					
9	Buffers: shared hit=6 read=22, temp read=10 written=12					
10	-> Sort	(cost=412.37..423.82 rows=4581 width=6)	(actual time=7.130..7.571 rows=4581.00 loops=1)			
11	Sort Key: i.film_id					

Total rows: 19 Query complete 00:00:00.092 CRLF Ln 22, Col 1

The screenshot shows the pgAdmin 4 interface with the same query as above. The query plan (QUERY PLAN) is displayed below the query text:

Step	Operation	Cost	Time	Rows	Width	Loops
9	Buffers: shared hit=6 read=22, temp read=10 written=12					
10	-> Sort	(cost=412.37..423.82 rows=4581 width=6)	(actual time=7.130..7.571 rows=4581.00 loops=1)			
11	Sort Key: i.film_id					
12	Sort Method: external merge Disk: 80kB					
13	Buffers: shared hit=6 read=22, temp read=10 written=12					
14	-> Seq Scan on inventory i	(cost=0.00..70.81 rows=4581 width=6)	(actual time=0.040..2.394 rows=4581.00 loops=1)			
15	Buffers: shared hit=3 read=22					
16	Planning:					
17	Buffers: shared hit=44					
18	Planning Time: 1.165 ms					
19	Execution Time: 22.146 ms					

Total rows: 19 Query complete 00:00:00.092 CRLF Ln 22, Col 1

- Structure: A sort node is present for the inventory table, sorting by i.film_id. The film table uses an index scan on film_pkey. PostgreSQL then performs a merge join on film_id.
- Compare:

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure: Servers (1) > PostgreSQL 18 > Databases (2) > dvdrental > postgres. The main window displays a SQL query in the 'Query' tab:

```
23 explain (analyze, buffers)
24 select f.film_id, f.title, i.inventory_id
25 from film f
26 join inventory i on f.film_id = i.film_id;
27
28 RESET ALL;
```

Below the query, the 'Data Output' tab shows the 'QUERY PLAN' for the executed query. The plan is as follows:

Step	Operation	Cost	Width	Rows	Actual Time	Actual Rows	Loops
1	Hash Join	110.50..193.39	23	4581	0.372..1.353	4581.00	1
2	Hash Cond: (i.film_id = f.film_id)						
3	Buffers: shared hit=113						
4	-> Seq Scan on inventory i	0.00..70.81	6	4581	0.017..0.198	4581.00	1
5	Buffers: shared hit=25						
6	-> Hash	98.00..98.00	19	1000	0.345..0.345	1000.00	1
7	Buckets: 1024 Batches: 1 Memory Usage: 59kB						
8	Buffers: shared hit=88						
9	-> Seq Scan on film f	0.00..98.00	19	1000	0.011..0.215	1000.00	1
10	Buffers: shared hit=88						
11	Planning:						
12	Buffers: shared hit=6						
13	Planning Time: 0.358 ms						

At the bottom, it shows 'Total rows: 14' and 'Query complete 00:00:00.104'. The status bar indicates 'CRLF' and 'Ln 25, Col 12'.

It switches back to hash join because, without order by and with the default settings restored, PostgreSQL considers hash join more efficient than merge join for these tables.